Course Outline: Understanding Telemetry from the Frontend

Title: Understanding Telemetry from the Frontend **Skill:** 43 — Collecting telemetry and context-related info from the user and your application **Learnpack:** + Entendiendo la telemetría desde el frontend **File:** s43-w15d43-entendiendo-la-telemetria-desde-el-front **Prerequisite:** Most Representative Cases in Telemetry (Skill 42 — industry case studies) **Target Audience:** Beginner developers in a full-stack AI bootcamp building production-grade frontend applications **Total Lessons:** 15 **Format:** Conceptual (100% conceptual — 13 📖 + 1 🤔 + 1 outro)

Course Description

This course answers the fundamental question every frontend developer faces when building a data-driven product: what should my application be sending to the backend, and why? Students learn a systematic taxonomy of the signals a frontend generates — from user identity and session data to behavioral events, performance metrics, and consent — and understand the specific objectives each category serves. The course culminates in the critical architectural distinction between user profile data (persistent state) and usage data (append-only events), which sets up the implementation work in the next learnpack.

Course Philosophy

This course emphasizes:

- Taxonomy over implementation: knowing WHAT to collect and WHY is the prerequisite for building it correctly
- Decision-first thinking: every data category is tied to a specific product objective
- Boundary clarity: precisely what belongs in each signal category to avoid confusion in the next learnpack's implementation

---

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Frontend Telemetry Overview	2
02 - Identity and Execution Context	4
03 - Behavioral and Quality Signals	3
04 - Why We Collect It	2
05 - Assessment	2

Section	Lessons
06 - Outro	1
Total	**15**

00.0 Welcome 🕮

Learning Objectives:

- Understand what this course covers: the taxonomy of frontend telemetry signals
- Recognize why knowing WHAT to collect and WHY is the necessary foundation before implementation
- Connect this course to the previous (industry cases showing telemetry value) and the next (implementing the sender)

Content Outline:

- The gap between knowing telemetry matters and knowing what to actually collect
- The question this course answers: given a frontend application, what signals should it send to the backend?
- Preview of the four signal categories: identity/context, behavioral, performance/quality, consent
- How this connects to the previous course (case studies showed the value) and the next (implementation)

Transitions: In the previous course, you saw how Spotify, Netflix, and Amazon use telemetry to make product decisions. This course gives you the signal vocabulary to build something similar in your own applications.

Key Principles:

- Collecting data without a purpose generates noise, not insight — every signal must connect to a decision
- The taxonomy here is not arbitrary: it reflects how production teams at real companies categorize their telemetry

Examples:

- A developer who knows "I should track user behavior" but doesn't know which specific events to capture will over-collect irrelevant events and miss the signals that matter
- Amazon's funnel optimization requires knowing which click events to capture before you can write the code that sends them

---

01.0 Telemetry from the Frontend 🕮

Learning Objectives:

- Define frontend telemetry and distinguish it from backend observability

- Understand the dual nature of frontend telemetry: user-generated signals AND application-generated signals
- Preview the six categories of data a frontend should collect

Content Outline:

- What "frontend telemetry" means: signals generated or observable from the client side
- Two distinct signal sources: the USER (what they do) and the APPLICATION (how it performs)
- Why frontend telemetry is different from backend observability: you are close to the human, so you capture intent and behavior, not just system events
- The six categories: identity/session, user profile, execution context, product usage events, performance, quality/support (plus consent as a governance layer)
- Preview: the next lesson maps these categories visually before deep-diving into each

Transitions: Backend observability tells you if your servers are healthy. Frontend telemetry tells you what humans are doing and experiencing. The next lesson gives you the complete signal map before we go section by section.

Key Principles:

- Frontend telemetry captures proxies for intent — a click is not just a click, it's a decision point
- Both user-generated signals (clicks, form submissions) and app-generated signals (Web Vitals, errors) belong in a frontend telemetry system
- Telemetry is a contract between your frontend and your data pipeline: undefined categories lead to schema chaos later

Examples:

- When a user submits a checkout form, three things happen simultaneously worth capturing: a user action (form submission), a potential performance signal (how long did the submission take?), and possibly an error signal (did it fail?)
- A Web Vital reading (LCP, FCP) is generated by the browser, not by the user — but it belongs in your telemetry payload alongside user actions

---

01.1 The Frontend Telemetry Signal Map 🕮

Learning Objectives:

- Name and describe all six signal categories a frontend should collect
- Explain at a high level what belongs in each category
- Understand consent as a cross-cutting governance constraint, not just another data field

Content Outline:

- The six categories as a map:
    1. Identity / Session — who is making the request, in this session
    2. User Profile — persistent attributes about who this user is
    3. Execution Context — where and how the app is running
    4. Product Usage Events — what the user does in the product

5. Performance — how fast the app is running
6. Quality / Support — what went wrong

- Consent as a governance layer that sits above all categories: only collect from categories the user has consented to
- How the categories relate to each other: identity anchors everything, profile enriches interpretation, context provides environment, events are the core signal, performance and quality are automatic monitors
- What the next three sections will cover: Sections 02-03 map the categories in detail

Transitions: The signal map gives you the vocabulary. Now we go section by section into what each category contains and how to populate it.

Key Principles:

- Every event payload should contain fields from ALL relevant categories — not just the event itself but its context (who did it, on what version, with what profile)
- Categories are not mutually exclusive: a single telemetry event often draws from identity, context, and product usage simultaneously
- Consent is not optional and cannot be treated as just another data field — it determines whether you may collect ANY of the other fields

Examples:

- A "checkout_submitted" event should include: userId (identity), appVersion (execution context), and the event-specific properties (checkout_total, payment_method) — not just the event name
- A company that separates "user profile fields" from "event fields" in their backend schema makes both querying and privacy compliance much easier

---

02.0 Identity and Execution Context 🕮

Learning Objectives:

- Understand the purpose of the Identity and Execution Context section
- Preview the three sub-categories: user identity/session, user profile signals, execution context signals
- Explain why identity and context are the envelope around every other signal

Content Outline:

- The envelope metaphor: every telemetry event is like a letter — the payload is the message, but you need an envelope (identity + context) to know where it came from and who sent it
- What this section covers:
  - User Identity and Session: who made this request (userId, sessionId, anonymousId)
  - User Profile Signals: persistent attributes about who this user is (role, plan, language, etc.)
  - Execution Context Signals: where and how the app is running (appVersion, env, route, featureFlags, etc.)
- Why these categories are grouped together: they are all context, not action — they don't describe what happened, they describe WHO and WHERE

- Preview: 02.1 covers identity, 02.2 covers profile, 02.3 covers execution context

Transitions: Identity and execution context are the metadata that makes all other signals interpretable. Without them, a "button_clicked" event is anonymous and decontextualized. The next three lessons populate this envelope category by category.

Key Principles:

- Identity data is immutable per session — once set at session start, it doesn't change within that session
- User profile data changes when the user changes (role upgrade, plan change) — it's state, not event
- Execution context data is set at app initialization and reflects the environment, not the user

Examples:

- Without userId, you cannot tell whether 1,000 users each experienced an error once, or 10 users each experienced it 100 times
- Without appVersion, you cannot tell whether a spike in errors was caused by a bug you just deployed

---

02.1 User Identity and Session 🕮

Learning Objectives:

- Identify the fields that constitute user identity and session data
- Explain the difference between authenticated identity (userId) and anonymous identity (anonymousId)
- Understand what a session is and why sessionId matters for analysis

Content Outline:

- The three identity fields:
  - `userId` — the authenticated user's permanent ID (only present after login)
  - `anonymousId` — a persistent identifier for unauthenticated users (stored in localStorage or cookie, survives page reloads, not sessions)
  - `sessionId` — a temporary identifier for the current session (reset on each new session, typically from a session cookie)
- Why anonymous tracking matters: users interact with your product before logging in (landing pages, onboarding flows, initial product exploration)
- Session vs User: a single user can have many sessions; a session groups the activity within one continuous visit
- What makes a session: typically a period of continuous activity; common definition = 30 minutes of inactivity ends a session
- The identity resolution problem: when an anonymous user logs in, their anonymousId events should be linked to their userId (identity stitching)

Transitions: Identity tells you WHO. But the same user on a free plan behaves differently from the same user on a paid plan. The next lesson covers the persistent profile attributes that give your analytics that context.

Key Principles:

- Never store email, phone, or other PII as the primary identity field — use opaque IDs that map to real user data only in your backend
- anonymousId should persist across sessions (stored in localStorage), not just within a session
- If you don't capture the anonymousId-to-userId mapping at login, you permanently lose the pre-login journey for that user

Examples:

- A user who reads your docs for 2 weeks before signing up has 2 weeks of anonymousId events. If you capture the mapping at signup, those events become part of their acquisition funnel story.
- sessionId lets you group all events within a single visit: the user went from landing page → pricing page → signed up. Without sessionId, you only see disconnected events.

02.2 User Profile Signals 🕮

Learning Objectives:

- Identify the fields that constitute user profile telemetry
- Explain the difference between profile data (who the user is) and event data (what the user does)
- Understand how profile signals enable personalization and segmentation in analytics

Content Outline:

- The user profile fields:
  - `role` / `persona` — what kind of user they are (admin, student, teacher, free, paid)
  - `plan` — their subscription tier (free, pro, enterprise)
  - `language` — their preferred language (for content personalization)
  - `timezone` — for time-relative analysis (when do they use the product?)
  - `country` — for geographic segmentation and compliance
  - `preferences` — theme (dark/light), notification settings, accessibility settings
- How profile fields are used: as dimensions for analysis (not just raw data)
  - "How do premium users behave differently from free users?" requires `plan` in every event
  - "Why do our Spanish users churn faster?" requires `language` in every event
- How profile data gets into events: typically attached at event collection time from application state, not fetched from the backend per event
- Profile data is NOT event data: it should be stored as application state (e.g., in a React context) and automatically included in every telemetry payload

Transitions: Profile tells you who the user is. Execution context tells you where and how they're using the app. The next lesson covers the third identity/context category.

Key Principles:

- Profile data should be set once per session at initialization (after auth) and attached to every subsequent event automatically
- Never capture profile data by re-fetching from the backend on every event — that's wasteful and slow; use application state
- Profile fields that change (plan upgrade, language switch) should trigger a profile update event as well as updating the cached profile in state

Examples:

- A "search_performed" event becomes significantly more valuable when it includes `role=student` and `plan=free` — you can now segment search behavior by user type
- Netflix's per-profile viewing data is powered by having `profile_id` attached to every playback event

---

02.3 Execution Context Signals 🕮

Learning Objectives:

- Identify the fields that constitute execution context telemetry
- Explain how execution context enables version-correlated debugging and feature flag analysis
- Understand why route and referrer data are considered context, not user actions

Content Outline:

- The execution context fields:
  - `appVersion` — the deployed version of the frontend (e.g., "2.4.1" or a commit hash)
  - `env` — the environment (production, staging, development)
  - `route` — the current page/path the user is on (e.g., "/dashboard", "/checkout")
  - `referrer` — where the user came from (previous page or external source)
  - `utm` parameters — marketing attribution (utm_source, utm_medium, utm_campaign)
  - `featureFlags` — which feature flags are enabled for this user/session
  - `tenant` / `academyId` — the organization context (for multi-tenant applications)
- How execution context is used:
  - `appVersion` — correlate bugs and performance regressions with deploys
  - `env` — filter out staging/dev events from production analytics
  - `route` — page-level analysis (which pages have the most errors? the most engagement?)
  - `featureFlags` — A/B test analysis (users with feature X enabled vs disabled)
- Execution context is set at initialization and on route changes — not per-event

Transitions: With identity and execution context covered, we move to the signals that describe what users actually DO — behavioral and quality signals.

Key Principles:

- `env` filtering is critical: never mix dev/staging events with production data — it corrupts your analytics
- `featureFlags` in telemetry is what makes A/B testing analysis possible: without it, you cannot separate "which users had the feature enabled" from raw event counts

- `appVersion` combined with error rate is how you know whether a new deployment broke something — without it, you're flying blind post-deploy

Examples:

- Without `appVersion`, a spike in errors after a deploy looks like a coincidence. With it, you can see "100% of these errors started at version 2.4.1."
- `featureFlags: { new_checkout_flow: true }` in every event from a user lets you later compare checkout completion rates for users with vs without the new flow

---

03.0 Behavioral and Quality Signals 🕮

Learning Objectives:

- Understand the purpose of the Behavioral and Quality Signals section
- Distinguish between behavioral signals (user-initiated), performance signals (app-generated), and quality/support signals (error state)
- Preview the two sub-lessons: product usage events, and performance/quality/consent

Content Outline:

- The two sub-categories:
  - Product Usage Events: the intentional actions a user takes (clicks, form submissions, searches, navigation)
  - Performance, Quality, and Consent: the automatic signals the app generates (Web Vitals, errors, consent status)
- Why they're grouped as "behavioral and quality": both describe what's happening in the product at runtime, from two perspectives — the user's intention and the system's health
- The complementary relationship: a user submitting a form (behavioral event) might coincide with an API error (quality signal) — together they tell the complete story of a failed interaction
- Preview: 03.1 covers product usage events, 03.2 covers performance, quality, and consent

Transitions: Identity and context set the stage. Behavioral and quality signals are the action — the running story of what's happening in your product.

Key Principles:

- Behavioral signals answer "what is the user trying to do?"; quality signals answer "is the product enabling them to succeed?"
- Both are needed: behavioral data without quality data hides whether users are succeeding; quality data without behavioral data loses the user's intent
- Consent governs both: if the user has not consented to analytics, you may not collect behavioral events

Examples:

- A user who clicks "Buy Now" three times in 30 seconds is probably experiencing a problem (quality signal: button click without success response) — behavioral data alone just shows 3 clicks, quality data explains why

- High engagement (many product usage events) combined with high error rate tells you users are trying hard but your product is failing them

---

03.1 Product Usage Events 🕮

Learning Objectives:

- Identify the key categories of user action events a frontend should capture
- Design a naming convention for product usage events
- Understand what properties to include in a usage event beyond the event name

Content Outline:

- The five usage event categories:
    1. Navigation events — page/route changes (routeChange, page_view)
    2. Key action events — high-value user actions (submit, enroll, search, checkout, copy)
    3. Engagement events — depth and duration signals (scroll_depth, time_on_page, video_play)
    4. Error interactions — when users encounter and respond to errors (error_dismissed, retry_clicked)
    5. Search events — what users look for (search_performed, search_result_clicked, search_no_results)
- Event naming conventions: use snake_case, object_action format
    - Good: `course_enrolled`, `checkout_submitted`, `search_performed`
    - Bad: `click`, `button_click`, `userClickedBuyButton`
- What to include in a usage event payload:
    - The event name (string)
    - The timestamp (ISO 8601)
    - Identity and context fields (from Section 02)
    - Event-specific properties (course_id, search_query, checkout_total, etc.)
- The signal-to-noise problem: not every click deserves an event — only capture events that connect to a product decision

Transitions: Usage events capture what users do intentionally. The next lesson covers the signals that happen automatically — performance, quality, and the consent layer that governs them all.

Key Principles:

- Name events by what they represent (user intent), not by what the UI element is (button_click is meaningless; checkout_submitted is actionable)
- Every event is a question the product team wants to answer — if no one will ever query this event, don't capture it
- Avoid capturing PII in event properties — use IDs, not names, emails, or phone numbers

Examples:

- Spotify's "track_skipped" event is a product usage event that powers their recommendation algorithm — without capturing WHEN in the track it was skipped, the data is significantly less useful

- Amazon's "product_viewed" event includes not just the product ID but also the referral path (how the user got to that page), making it far more valuable than just a page view

Anti-pattern — Over-capturing: Adding a telemetry event for every button click is tempting but counterproductive. It creates enormous event volumes, makes the schema unstable (buttons change), and produces data no one analyzes. The correct approach: identify the 10-20 user actions that directly connect to product decisions, and capture only those with rich context. You can always add more later; you cannot un-clutter an over-tracked dataset.

03.2 Performance, Quality, and Consent 🕮

Learning Objectives:

- Identify the performance signals a frontend should capture and explain their relationship to Skill 35 (Core Web Vitals)
- Identify the quality/support signals (error state, network state, API status)
- Explain how consent signals work as a governance layer over all other telemetry

Content Outline:

- Performance signals:
  - Web Vitals as telemetry: FCP, LCP, TBT (covered in Skill 35 as optimization targets — here they're telemetry signals sent to the backend)
  - API latency: how long do your API calls take? (measured at the client, not the server)
  - CSR/SSR timing: how long does the page take to become interactive?
  - Re-render counts (if applicable): excessive re-renders as a performance indicator
  - Key distinction: in Skill 35 these were metrics to optimize against; here they are signals to COLLECT and send to your analytics backend
- Quality / Support signals:
  - `errorId` and `stack` (sanitized): when an error occurs, capture a unique error ID and a sanitized stack trace
  - `networkState`: is the user online, offline, or on a slow connection? (navigator.onLine, connection.effectiveType)
  - `apiStatus`: the HTTP status codes your API calls are returning (to correlate user behavior with backend failures)
- Consent signals:
  - `consent.analytics`: whether the user has consented to analytics tracking
  - `consent.marketing`: whether the user has consented to marketing tracking
  - `consent.functional`: whether the user has consented to functional cookies/tracking
  - How consent works as a gate: if `consent.analytics=false`, no analytics events may be fired
  - Consent signals are DATA to collect (store the user's consent state), not just policy constraints

Transitions: You now know the full signal taxonomy — what to collect. The next section shifts to WHY: the specific objectives each category serves.

Key Principles:

- Web Vitals that you optimize against (Skill 35) and Web Vitals you send as telemetry signals are the same numbers — the difference is that telemetry sends them to your backend where you can track them over time across all users
- Quality signals are most valuable when combined with behavioral signals: an error alone tells you something broke; an error preceded by 3 retries of the same action tells you it's a critical path
- Consent data is both policy (what you're allowed to collect) and product data (what percentage of your users have consented — a product and legal metric)

Examples:

- Netflix monitors API latency from the client side (not just the server) because the full round-trip matters for user experience — server-only latency measurements miss the last-mile problem
- A support team with access to `errorId`, `networkState`, and the user's `sessionId` can reproduce a reported bug without asking the user a single question

---

04.0 Why We Collect It 🕮

Learning Objectives:

- Name and describe the five primary objectives for collecting frontend telemetry
- Map specific signal categories to the objectives they serve
- Understand why collecting data without a clear objective creates waste

Content Outline:

- The five objectives of frontend telemetry:
    1. Personalization — content and UX adaptation based on role, language, and preferences
    2. Product Analytics — understanding user journeys, feature adoption, drop-offs, and cohort behavior
    3. Observability — detecting errors, performance degradation, and incidents by version
    4. Experimentation — A/B testing and feature flag impact measurement (which variant converts better?)
    5. Support — reproducing reported bugs using session context, breadcrumbs, and error state
- Mapping signals to objectives:
    - Personalization: user profile (role, language, plan, preferences)
    - Product Analytics: product usage events, navigation events, session data
    - Observability: performance signals, quality/support signals, appVersion
    - Experimentation: featureFlags, product usage events, conversion metrics
    - Support: errorId, stack, networkState, sessionId, breadcrumbs (recent events)
- The "collect with purpose" principle: before adding a new signal, ask "which objective does this serve?"

Transitions: You now know what to collect and why. The final content section before the assessment covers the architectural distinction between two fundamentally different kinds of data: user profile state and usage events.

Key Principles:

- Every signal in your telemetry schema should trace back to at least one of these five objectives
- Signals that serve multiple objectives are the most valuable (e.g., featureFlags serves both experimentation AND observability)
- Signals that serve NO objective are noise — and noise degrades the signal-to-noise ratio of your entire analytics pipeline

Examples:

- Google Analytics' "sessions" metric is product analytics data; their "Core Web Vitals" is observability data. Same tool, two different objectives.
- A bug report that says "the checkout failed" with errorId, sessionId, and the user's last 10 events (breadcrumbs) is fully reproducible; the same report without those fields requires a 30-minute back-and-forth with the user

---

04.1 User Profile vs Usage Data 🕮

Learning Objectives:

- Clearly distinguish between user profile data (persistent state) and usage data (append-only events)
- Explain the architectural implications of the distinction: different storage, different update patterns, different query patterns
- Understand why confusing the two leads to schema problems and data corruption

Content Outline:

- User Profile data: persistent state about who the user is
  - Stored as a record in your database (typically in a `users` or `profiles` table)
  - Updated via PATCH when the user changes their profile (e.g., `PATCH /users/me`)
  - Never appended to — the record is updated in place
  - Examples: name, role, plan, language, timezone, preferences
- Usage data: append-only events about what the user does
  - Stored as events in your telemetry store (time-series DB, event queue, data warehouse)
  - Never updated — each event is immutable once written
  - Accumulated over time — the full history is preserved
  - Examples: page_view, course_enrolled, checkout_submitted, error_occurred
- Why the distinction matters architecturally:
  - Querying "what is the user's current role?" → profile table (single row, current state)
  - Querying "how did this user's behavior change when their role changed from free to pro?" → join profile events with usage events on timestamp
  - If you store role changes as mutable updates to the profile (no history), you lose the ability to answer the second question
- The practical rule: if it describes who the user IS → profile state. If it describes what the user DID → usage event.

Transitions: This distinction is the conceptual foundation for how you'll model your telemetry schema in the next learnpack. Now test your understanding of the full taxonomy.

Key Principles:

- Profile and usage data should live in different storage: profile in your transactional DB, usage in your event store
- A profile "update" event is BOTH: you update the profile record (persistent state) AND emit a `profile_updated` event (append-only event documenting the change)
- Sending profile data as events instead of state updates is a common mistake that makes real-time personalization impossible (you'd have to replay all events to get current state)

Examples:

- If a user changes their plan from "free" to "pro", you do two things: (1) PATCH /users/me with `{plan: "pro"}` to update their profile, and (2) emit a `plan_upgraded` event with the old and new plan. Now you have both current state AND history.
- Airbnb's pricing algorithm needs the user's CURRENT booking preferences (profile) for immediate decisions, and their HISTORICAL booking behavior (usage events) for pattern analysis — these require different data structures

---

05.0 From Signal to Decision 🕮

Learning Objectives:

- Connect the full signal taxonomy to the product decisions it enables
- Identify which signals enable which decisions in realistic product scenarios
- Articulate why the signal taxonomy (WHAT and WHY) must be established before implementation (HOW)

Content Outline:

- Closing the loop: how signals become decisions
  - Signal collected → stored in event pipeline → aggregated in analytics → viewed in dashboard or fed to ML model → decision made → product change → new signals generated
- Decision examples by signal category:
  - "Which features should we build next?" → product usage events showing which parts of the app are most used
  - "Why are European users churning faster?" → user profile (country) + usage events (churn event) + product behavior (what they did before churning)
  - "Did the new checkout flow improve conversion?" → featureFlags + checkout_submitted events + funnel analysis
  - "What caused the outage?" → appVersion + error signals + API latency correlated with deploy timestamp
  - "Why can't this user's bug be reproduced?" → sessionId + errorId + last 10 events (breadcrumbs)
- Why WHAT and WHY must precede HOW:

    - Without knowing what you'll collect, you cannot design the event schema
    - Without knowing why (objective), you don't know what context to attach to each event
    - The next learnpack implements the sender — but it will only work well if you've designed the signal taxonomy first
- The next learnpack: implementing the event schema and the sender function

Transitions: Test your understanding of the signal taxonomy before moving to the implementation.

Key Principles:

- The signal taxonomy is a specification document before it's a codebase — your data team and frontend team should agree on it before writing a single line of telemetry code
- Every signal you collect has a downstream cost (storage, processing, privacy compliance) — that cost is justified only by the decisions it enables
- Starting with the decision ("I want to know if users are churning by plan tier") and working backward to the signals ("I need plan in every event and a churn event") is more effective than collecting everything and hoping to find insights later

Examples:

- Spotify's "skip within 10 seconds" insight required: `track_skipped` event (product usage) + `skip_timestamp` property + `track_duration` property. The insight only exists because someone decided in advance that skip timing mattered.
- Google's 53% bounce rate statistic for mobile sites that load in 3+ seconds required: LCP measurement (performance signal) + session_abandoned event (behavioral signal). Two signals, one powerful insight.

05.1 Frontend Telemetry Concepts 🧠

Learning Objectives:

- Apply the signal taxonomy to new scenarios
- Map signals to objectives in context
- Distinguish between profile data and usage data in realistic examples

Question Format: Scenario-based

Questions:

1. A product manager says: "We want to understand why users on the free plan rarely use our most powerful features." List the specific telemetry signals (with field names) you would need to answer this question, and identify which signal category each comes from.

2. A developer adds a telemetry event called `click` with no additional properties. What is wrong with this event design, and how would you improve it? Rewrite it following the naming conventions and property standards from this course.

3. Your application logs `consent.analytics = false` for 40% of your users. What happens to your product analytics data? How should your telemetry system respond to this consent status,

and what are the implications for your analytics dashboards?

4. A user reports: "I tried to enroll in a course three times and it didn't work." With access to your telemetry system, what specific signals would you use to reproduce and diagnose this issue, and from which signal categories do they come?

5. A company stores user plan changes by overwriting the `plan` field in their users table (no history). A month later, the analytics team asks "how do users' behavior patterns change in the first week after upgrading from free to pro?" Can they answer this question? Why or why not? What should they have done differently?

6. Match each product objective to the primary signal category that enables it: (a) Detecting whether a bug is caused by a specific app version → ___; (b) Personalizing the interface language for a returning user → ___; (c) Measuring the impact of a new checkout flow on conversion rate → ___; (d) Reproducing a user's reported crash → ___.

Evaluation Criteria:

- Q1: Does the student name specific field names (not just categories)? Does the answer include both profile fields (plan) and event fields (feature_used)?
- Q2: Does the student identify both the naming problem AND the missing context (no properties)?
- Q3: Does the student correctly identify that all analytics events must be suppressed, and that dashboards based on 100% of users will now reflect only 60%?
- Q4: Does the answer include errorId, sessionId, quality signals, AND the behavioral events (course_enroll_clicked)?
- Q5: Does the student identify that history is lost and that the solution is to emit profile_updated events alongside the state mutation?
- Q6: (a) execution context (appVersion), (b) user profile (language), (c) feature flags + usage events, (d) quality signals (errorId) + session (sessionId)

Score Interpretation:

- 5-6 correct: Strong grasp of the signal taxonomy and its practical application
- 3-4 correct: Good foundation; review the signal-to-objective mapping
- 0-2 correct: Return to the signal categories and the user profile vs usage data distinction

---

06.0 Outro

Content Outline:

- Course recap: what students accomplished
  - Learned the complete taxonomy of frontend telemetry signals: identity/session, user profile, execution context, product usage events, performance/quality, and consent
  - Mapped each category to the product objectives it enables
  - Understood the critical architectural distinction between persistent profile state and append-only usage events
- Connection to bigger picture

    - This course was the specification phase — you now know WHAT to collect and WHY
    - The next learnpack (same Skill 43) implements the sender: how to format events as JSON, batch and send them to your backend, and handle reliability edge cases
    - After implementation comes capture (Next.js-specific hooks) and then guardrails (consent enforcement, PII scrubbing, allowlists)
- Next steps and continued learning
    - Amplitude's Event Taxonomy guide: real-world example of how a product analytics company structures its event schema
    - Segment's documentation on tracking plans: how production teams manage signal taxonomy as a team artifact
    - privacy.gg: interactive GDPR/CCPA compliance checker for telemetry decisions
- Encouragement
    - Most developers skip the taxonomy phase and build the sender first — then spend weeks refactoring when they realize they're collecting the wrong things. You're starting from a solid foundation.

Note: This is conclusion only — no theory, exercises, or assessment.